

# Objektorientierte *Analyse*

# Inhalt

- System & Akteurdefinition
- Ablaufpläne
- Von der Analyse zum Design

# Von der Aufgabenstellung zum Design

- Wir wollen uns damit befassen, wie wir von der Aufgabenstellung bis zum Design kommen. Hierbei betrachten wir mehrere Schritte von Anforderungsanalyse bis zum fertigen Design/Pflichtenheft
- Um die Schritte zu verdeutlichen, arbeiten wir an einem Beispiel
- Beispiel aus Vorlesungsskript “Objektorientiertes Programmieren” von Bernd J. Krämer.

# Adas Blumenladen

In Adas Blumenladen von Inhaberin Ada König soll ein Verwaltungssystem eingeführt werden, welches verschiedene Geschäftsprozesse Verkauf, Bestellung, und Lieferung unterstützen soll.

In Adas Blumenladen arbeiten neben der Chefin Frau König noch weitere VerkäuferInnen. Das Sortiment umfasst Pflanzen, Zubehör, und Dekoartikel.

Zusätzlich zu regulären Kunden gibt es Premiumkunden, die eine Kundennummer besitzen. Frau König und alle anderen Verkäufer bedienen Kunden und erfassen Premiumkunden.

# Adas Blumenladen

Alle können im Laden bezahlen, bar oder mit Kreditkarte. Bei Beträgen ab 100 Euro kann auch per Rechnung bezahlt werden. Sollte eine Rechnung nicht innerhalb von 14 Tagen beglichen werden, muss Frau König eine Mahnung versenden.

PremiumkundInnen können telefonisch eine Lieferung in Auftrag geben. Lieferungen können immer auf Rechnung bezahlt werden. Vor dem (kostenlosen) Versand durch einen Kurier muss die Rechnung beglichen sein.

# Adas Blumenladen

Neben dem Verkauf muss auch das Lager gefüllt werden, sollten die Vorräte aufgebraucht sein. Dies kann entweder bei einer regelmäßigen Überprüfung auffallen, oder aber, wenn bei einem Kundenwunsch festgestellt wird, dass nicht genug Artikel vorhanden sind.

Die Bestellung muss durch die Chefin erfolgen, ein Verkäufer kann lediglich das frühestmögliche Lieferdatum erfragen.

Weiterhin kann die Chefin Artikel ins Sortiment aufnehmen, oder diese entfernen.

# Entwicklung der Fallanalyse

Wir wollen nun

- die Handlungsträger identifizieren
- ihr Zusammenwirken beschreiben
- die für die Abläufe wesentlichen Handlungen erkennen
- Informationsaustausch nachvollziehen
- Gegenstände, mit denen die Handlungsträger umgehen identifizieren
- die Beziehungen zwischen den Gegenständen beschreiben
- und abschließend Ideen diskutieren, wie die Geschäftsvorfälle durch den Computer unterstützt werden können

# Anwendungsfall & Akteure

Ein Anwendungsfall (engl. use case) bezeichnet eine typische Interaktion zwischen einem Akteur und einem System aus der Sicht des Akteurs. Ein Anwendungsfall umfasst Folgen von Aktionen, die reguläre, aber auch vom normalen Verhalten abweichende Abläufe beschreiben.

Anwendungsfälle erfassen genau die Systemaktionen, die für das Zusammenwirken zwischen dem System und den Akteuren erforderlich sind und dazu beitragen, bestimmte Ziele der Akteure zu erreichen.

Anwendungsfälle werden durch Akteure oder durch Zeitereignisse angestoßen.

# Anwendungsfall & Akteure

Mögliche Anwendungsfälle aus Adas Blumenladen

- Bestellung beim Großhändler
- Erfassung eines Premiumkunden
- Verkauf eines Artikels

Die Anwendungsfälle werden durch Mitarbeiter oder durch Kunden angestoßen.

# Anwendungsfall & Akteure

Anwender und externe Systeme, die an Anwendungsfällen teilnehmen, aber selbst nicht Teil des zu realisierenden Systems sind, werden als Akteure (engl. actor) bezeichnet.

Mögliche Akteure in unserer Fallstudie sind Großlieferanten, die Chefin, Verkäufer, Kunden und Premiumkundinnen sowie der Kurier.

# Anwendungsfall & Akteure

Bei der Auswahl der Akteure müssen wir folgendes beachten:

Zwei Akteure unterscheiden sich dadurch, dass die unterschiedliche Rollen haben, nicht dadurch, dass sie unterschiedliche Personen sind. Es kann sein, dass dies mit verschiedenen Personen übereinstimmt, dies ist jedoch nicht zwingend erforderlich.

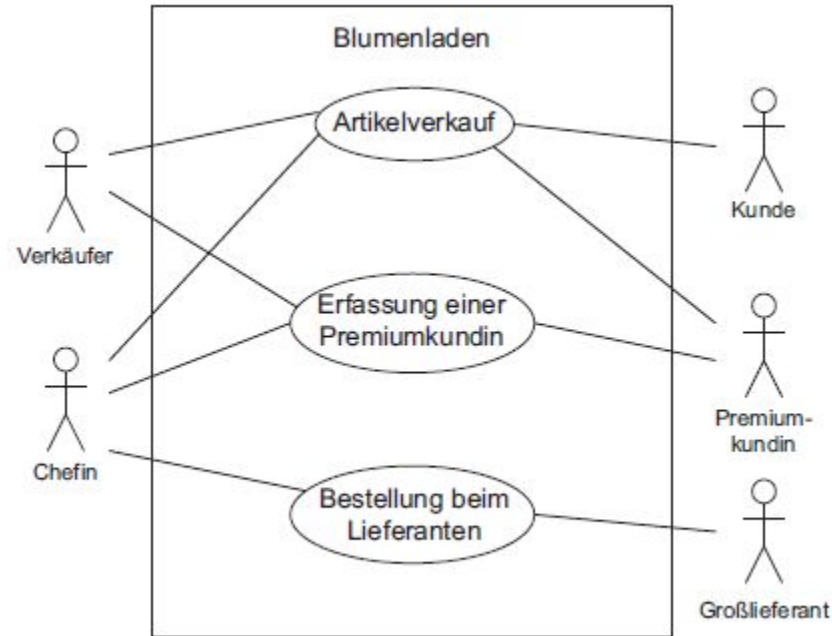
In unserem Beispiel ist Frau König (die Chefin) ein Akteur. Die anderen VerkäuferInnen sind ebenfalls Akteure, die jedoch nicht weiter unterschieden werden.

# Anwendungsfall & Akteure

Es gibt lediglich eine Beziehung zwischen Akteuren und Anwendungsfall: Der Akteur nimmt an diesem teil.

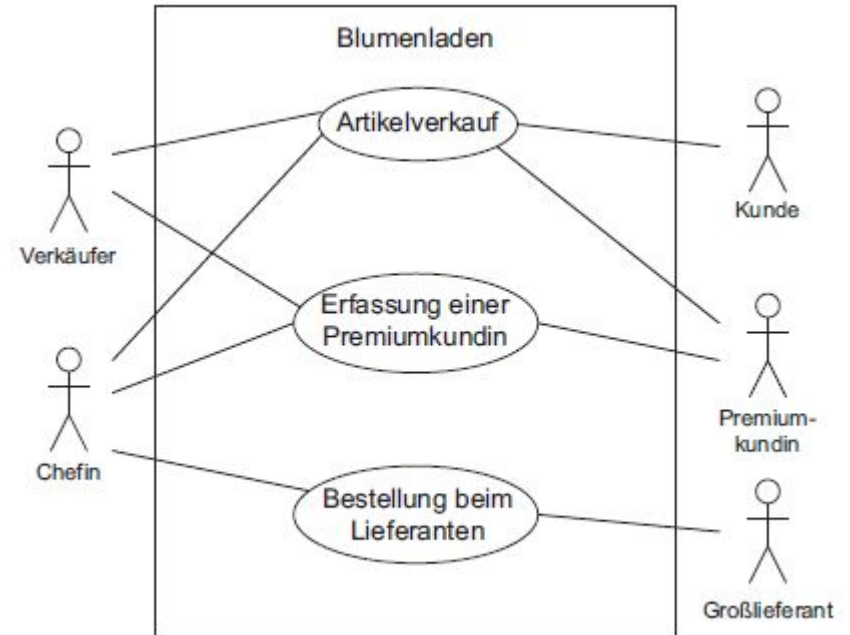
Eine weitere Beziehung gibt es nicht. Sobald der Akteur etwas mit diesem Anwendungsfall zu tun hat, nimmt er teil.

# Darstellung der Akteure und Anwendungsfälle (teilweise)



# Darstellung der Akteure und Anwendungsfälle (teilweise)

- Akteure werden durch Strichmännchen beschrieben, sie stehen außerhalb des Systems
- Wir haben hier lediglich einen Teil der Anwendungsfälle angegeben, das Lager fehlt
- Es ist nicht nötig, alle Anwendungsfälle in ein Diagramm zu bringen



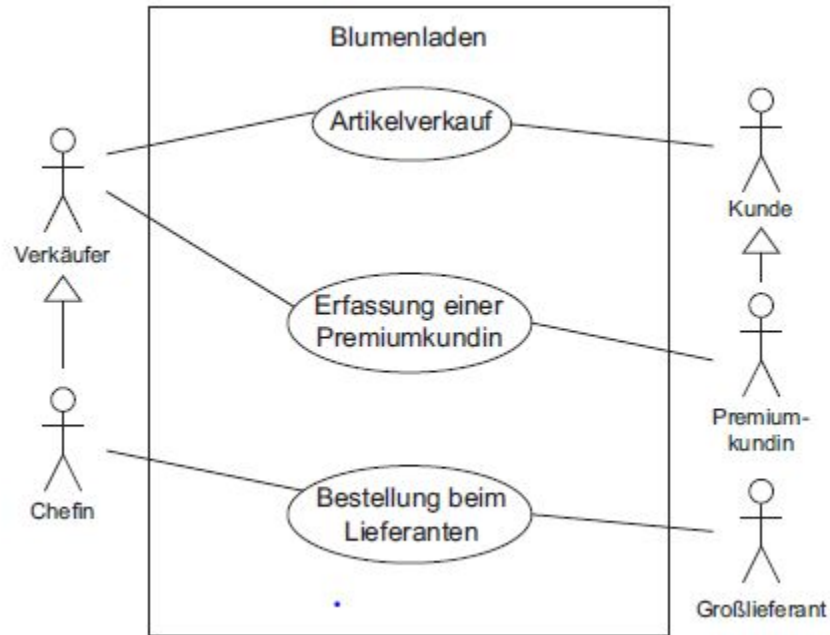
# Beziehungen zwischen Akteuren

So wie bei Klassen müssen wir auch hier Beziehungen durch Spezialisierungen angeben, um keine unnötigen Beziehungen zu erzeugen.

Ein Akteur kann einen anderen Akteur erweitern, wenn er dessen Rolle vollständig ausfüllt, aber noch an weiteren Anwendungsfällen beteiligt ist.

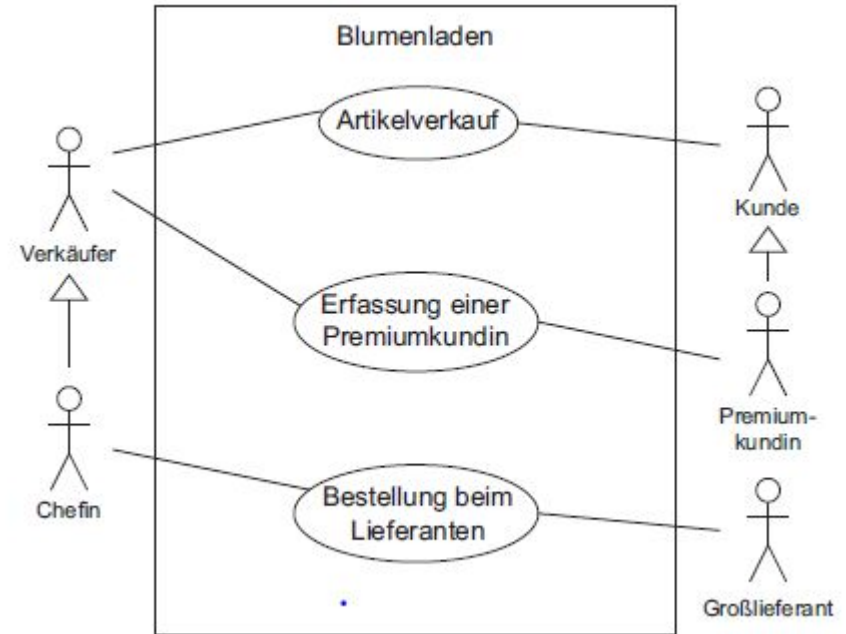
Beachte, dass dies nicht der Fall ist, falls der Akteur auf andere Art und Weise am gleichen Anwendungsfall beteiligt ist.

# Beziehungen zwischen Akteuren



# Beziehungen zwischen Akteuren

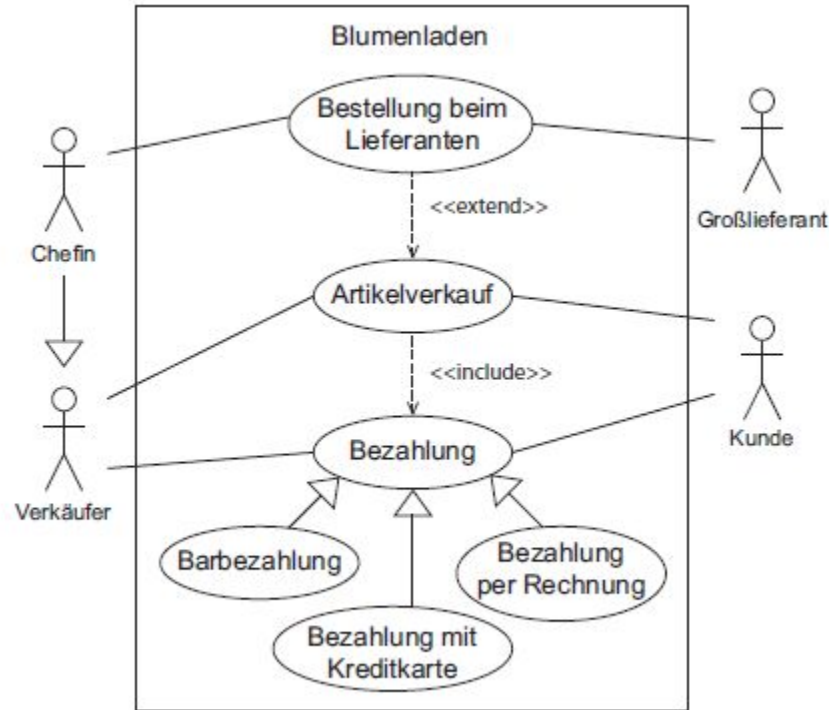
- Die Chefin verhält sich wie ein Verkäufer, hat jedoch weitere Anwendungsfälle, an denen sie teilnimmt.
- Weiterhin ist ein Premiumkunde ein Kunde mit weiteren Anwendungsfällen.



# Beziehungen der Anwendungsfälle

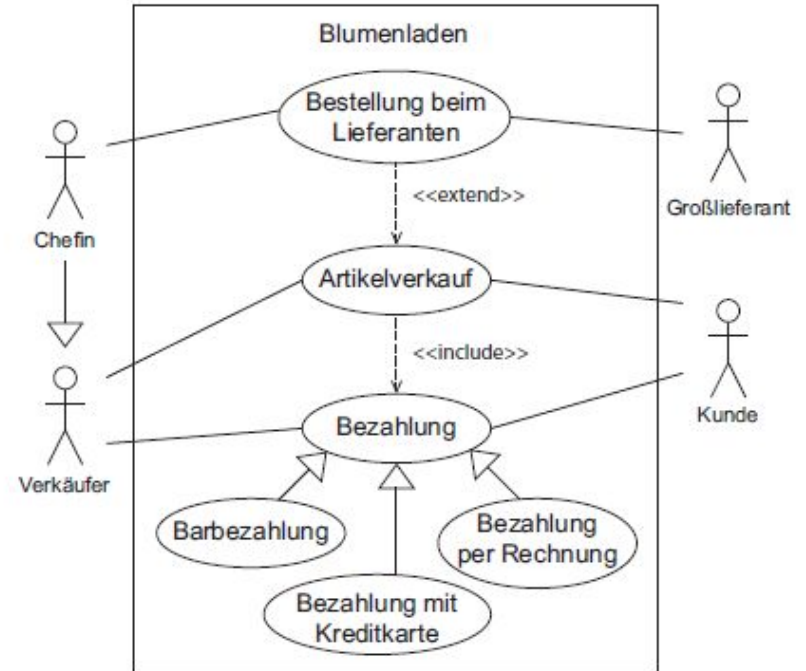
- Wir unterscheiden zwei Arten von Beziehung von Anwendungsfällen: Enthalten und Erweitern.
- Ein Anwendungsfall enthält einen Anwendungsfall, wenn er in jedem Fall ausgeführt werden muss, zum Beispiel das Bezahlen bei einem Verkauf.
- Ein Anwendungsfall erweitert einen einen Anwendungsfall, falls es unter Umständen ebenfalls (nach) dem aktuellen Anwendungsfall ausgeführt werden muss. Zum Beispiel kann ein Verkauf zu einer Bestellung führen.

# Beziehungen zwischen Anwendungsfällen



# Beziehungen zwischen Anwendungsfällen

- Bestellung ist ein optionaler Vorgang, er kann von anderen Akteuren ausgeführt werden (auch abstrakt anderen).
- Bezahlung ist ein Pflichtvorgang. Er wird vom gleichen konkreten Akteur vorgenommen.



# Szenario

Wir nennen eine einzelne Abfolge von Aktionen, d. h. einen Ablaufpfad in einem Anwendungsfall, ein Szenario.

Ein Szenario, das die Sicht der Anwender wiedergibt, fasst das zu entwickelnde System als Blackbox auf. Die innere Arbeitsweise des Systems bleibt den Anwendern verborgen, von Interesse ist allein das externe Verhalten. Das Augenmerk liegt bei dieser Perspektive auf typischen Interaktionen zwischen Benutzern und dem Anwendungssystem.

# Szenario

- Szenarien sind die wichtigsten Hilfsmittel bei Design und Implementation eines Systems
- Sie helfen dem Entwickler, die erwartete Funktionsweise des Systems zu verstehen
- Es sollten immer eher zu viele als zu wenige Szenarien beschrieben werden.
- Wir beschreiben ein Szenario in einer Tabelle, deren Felder wir im Folgenden herausarbeiten wollen
- Jeder Anwendungsfall braucht mindestens ein Szenario, kann aber mehrere enthalten

# Szenario

- Wir geben dem Anwendungsfall einen Namen, und beschreiben ihn kurz.
- Anschließend führen wir die Akteure ein, und geben ihnen Kürzel zur späteren Referenz.

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| Anwendungsfall     | Bezeichnung                                             |
| Kurzbeschreibung   | Wesentliche Funktionen sowie Bedeutung für den Anwender |
| Beteiligte Akteure | Kürzel und Bezeichnung                                  |

# Szenario

|                  |                                                                                                                                                                                                                                                       |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Anwendungsfall   | Bestellung eines Premiumkunden                                                                                                                                                                                                                        |
| Kurzbeschreibung | Ein Premiumkunde kann ein Bestellung aufgeben und diese dann kostenlos nach Hause liefern lassen. Die Bestellung kann telefonisch oder persönlich erfolgen. Es sind sowohl Barbezahlung als auch Bezahlung mit Kreditkarte oder per Rechnung möglich. |

# Szenario

|                    |                |                                     |
|--------------------|----------------|-------------------------------------|
| Beteiligte Akteure | Vk<br>Pk<br>Ku | Verkäufer<br>Premiumkunde<br>Kurier |
|--------------------|----------------|-------------------------------------|

# Szenario

- Anschließend geben wir Vor- und Nachbedingungen an.
- Vorbedingungen geben an, welcher Systemzustand für erfolgreiche Ausführung vorhanden sein muss.
- Nachbedingungen geben an, welcher Systemzustand nach der Ausführung gelten muss.

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| Vorbedingung    | Systemzustand, der für erfolgreiche Ausführung erforderlich ist |
| Nachbedingungen | Systemzustand, der nach erfolgreicher Ausführung vorliegt       |

# Szenario

|               |                                                                                                        |
|---------------|--------------------------------------------------------------------------------------------------------|
| Vorbedingung  | Der bestellte Artikel muss auf Lager oder rechtzeitig bestellbar sein. Der Kunde ist ein Premiumkunde. |
| Nachbedingung | Der Artikel wurde an den Kunden geliefert und der entsprechende Betrag bezahlt.                        |

# Szenario

- Wir geben einen Auslöser an, also was passieren muss, damit dieses Szenario aufgerufen wird.

|          |                                                                                                                                           |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Auslöser | Ereignisse oder Aktionen, die den Anwendungsfall auslösen, z. B. zeitliche Ereignisse oder auch Ausnahmefälle in anderen Anwendungsfällen |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------|

# Szenario

|          |                                                    |
|----------|----------------------------------------------------|
| Auslöser | Anruf eines Kunden oder Besuch des Kunden im Laden |
|----------|----------------------------------------------------|

# Szenario

- Der wichtigste Teil der Tabelle ist der Ablauf
- Wir beginnen mit dem Standardszenario, und geben anschließend Alternativen und Erweiterungen an
- Der Ablauf wird linear in Schritten angegeben, wobei die jeweiligen Akteure genannt werden

|                                           |                                                                                                                                                                                                                                                                                                |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Standardszenario<br><br>Schrittnr, Akteur | Kurzbeschreibung<br>der einzelnen<br>Schritte eines<br>erfolgreichen<br>Ablaufs, der<br>entweder mit der<br>auslösenden Aktion<br>oder dem Eintreten<br>der<br>Vorbedingung<br>beginnt und mit dem<br>gewünschten<br>Ergebnis oder dem<br>Eintritt der<br>erwarteten<br>Nachbedingung<br>endet |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# Standardszenario

|    |    |                                                                                    |
|----|----|------------------------------------------------------------------------------------|
| 1. | Pk | Der Kunde nennt die gewünschten Artikel persönlich im Laden.                       |
| 2. | Vk | Der Verkäufer stellt fest, dass die Artikel in ausreichender Menge auf Lager sind. |
| 3. | Pk | Der Premiumkunde nennt gültige Kundennummer, einen Liefertermin und Adresse.       |
| 4. | Vk | Verkäufer erfasst die Daten und berechnet den Preis.                               |
| 5. | Pk | Der Kunde bezahlt bar (Anwendungsfall Barbezahlung).                               |
| 6. | Ku | Der Kurier liefert die gewünschten Artikel am definierten Datum.                   |

# Alternative Szenarien

- Alternative Szenarien können Abweichungen angeben
- Hierbei wird jeweils in einem Schritt abgewichen
- Vor- und Nachbedingung dürfen sich nicht ändern (siehe Fehlersituationen)

|                       |                                                                       |
|-----------------------|-----------------------------------------------------------------------|
| Alternative Szenarien | z. B. Redefinition oder Alternativen für Schritte im Standardszenario |
| Schrittnr, Akteur     |                                                                       |

# Alternative Szenarien

|       |    |                                                                                                |
|-------|----|------------------------------------------------------------------------------------------------|
| Zu 1. | Pk | Der Kunde nennt die gewünschten Artikel telefonisch.                                           |
| Zu 5. | Vk | Der Kunde nennt seine Kreditkartennummer an Telefon (Anwendungsfall Bezahlung mit Kreditkarte) |
| Zu 5. | Vk | Der Kunde legt seine Kreditkarte vor (Anwendungsfall Bezahlung mit Kreditkarte)                |
| Zu 5. | Vk | Der Kunde zahlt beim Eintreffen der Ware (Anwendungsfall Barzahlung)                           |

# Erweiterungen

- Erweiterungen können zusätzliche Abläufe beschreiben
- Sie ändern ebenfalls die Nachbedingung nicht

|                                    |                                                   |
|------------------------------------|---------------------------------------------------|
| Erweiterungen<br>Schrittnr, Akteur | Erweiterung von<br>Schritten im<br>Standardablauf |
|------------------------------------|---------------------------------------------------|

# Erweiterungen

|       |    |                                                                                                                                                                                            |
|-------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zu 5. | Vk | Artikel sind nicht in ausreichender Menge vorhanden, können aber rechtzeitig geliefert werden. Die Chefin muss eine Bestellung veranlassen<br>(Anwendungsfall Bestellung beim Lieferanten) |
|-------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# Fehlersituationen

- Fehler können den erfolgreichen Ablauf verhindern
- Die Nachbedingungen werden nicht erfüllt
- Eine Fehlersituation beschreibt daher nicht nur den Fehler, sondern auch den neuen Status des Systems

|                                          |                                                                                                                                                               |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fehlersituation<br><br>Schrittnr, Akteur | Situationen, die bei diesem Schritt zu einer nicht erfolgreichen Ausführung führen. Beschreibung des Systemzustands nach einer nicht erfolgreichen Ausführung |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|

# Fehlersituationen

|       |    |                                                                                                                                                                                                                      |
|-------|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zu 2. | Vk | Artikel ist nicht in ausreichender Menge vorhanden, kann auch nicht rechtzeitig geliefert werden; Anwendungsfall wird abgebrochen, aber Bestellung trotzdem vorgenommen (Anwendungsfall Bestellung beim Lieferanten) |
| Zu 3. | Pk | Kunde nennt ungültige Kundennummer; Anwendungsfall wird abgebrochen                                                                                                                                                  |
| Zu 5. | Pk | Der Kunde bezahlt nicht rechtzeitig; Lieferung findet nicht statt bzw. wird im Falle einer fehlgeschlagenen Bezahlung vor Ort wieder mitgenommen                                                                     |

# Übersicht Tabelle

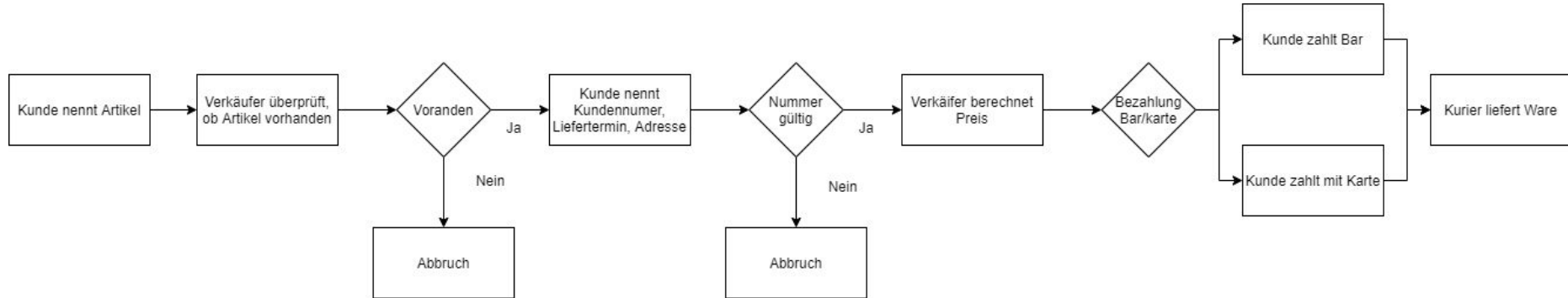
|                       |
|-----------------------|
| Anwendungsfall        |
| Kurzbeschreibung      |
| Beteiligte Akteure    |
| Vorbedingungen        |
| Nachbedingungen       |
| Auslöser              |
| Standardszenario      |
| Alternative Szenarien |
| Erweiterungen         |
| Fehlersituationen     |

# Was ist zu fragen?

- Warum benutzt der Akteur das System?
- Welche Art von Antwort erwartet der Akteur von der Aktion?
- Was muss der Akteur tun, um das System benutzen zu können?
- Welche Informationen muss der Akteur dem System übermitteln?
- Welche Informationen erwartet der Akteur als Antwort vom System?

|                       |
|-----------------------|
| Anwendungsfall        |
| Kurzbeschreibung      |
| Beteiligte Akteure    |
| Vorbedingungen        |
| Nachbedingungen       |
| Auslöser              |
| Standardszenario      |
| Alternative Szenarien |
| Erweiterungen         |
| Fehlersituationen     |

# Alternative Darstellung: Diagramm



# Alternative Darstellung: Diagramm

- In der Darstellung als Diagramm kann man Alternativen direkt einbauen
- Diese Darstellung sollte ergänzend gewählt werden, nicht ausschließlich
- Die Form kann UML folgen, wichtig ist jedoch, dass Informationen korrekt transportiert werden
- Wenn nicht gerade externe Firmen ohne weitere Kommunikation während der Implementation beauftragt werden sollen, ist Kommunikation oft wichtiger als korrektes UML

Tools: Draw.io, [Mermaid.js](#), [Online Editor für Mermaid](#)

# Von der Analyse zum Design

Bisher haben wir uns damit befasst, die Probleme des Kunden zu analysieren.

Wir wollen nun die Ablaufpläne nehmen und damit ein Design für eine Software entwickeln.

Die Kollektion der Analyse wird auch als “Lastenheft” bezeichnet, das Design als “Pflichtenheft”.

# Was ist das Design

Das Design beinhaltet den kompletten Bauplan für die fertige Software.

Mit Hilfe des Designs sollte der Entwickler die Software implementieren können.  
Die vorher erzeugten Ablaufpläne dienen hauptsächlich dem Erstellen des Designs, können aber als Hilfe herangezogen werden.

Das Design sollte am Ende vollständig und eindeutig sein.

# Was beinhaltet das Design

- Unterteilung der Software in funktionale Blöcke (Module)
- Eine genaue Beschreibung eines jeden Moduls
- Die Interaktion der Module
- Die Oberfläche der Software

# Adas Blumenladen

In Adas Blumenladen von Inhaberin Ada König soll ein Verwaltungssystem eingeführt werden, welches verschiedene Geschäftsprozesse Verkauf, Bestellung, und Lieferung unterstützen soll.

In Adas Blumenladen arbeiten neben der Chefin Frau König noch weitere VerkäuferInnen. Das Sortiment umfasst Pflanzen, Zubehör, und Dekoartikel.

Zusätzlich zu regulären Kunden gibt es Premiumkunden, die eine Kundennummer besitzen. Frau König und alle anderen Verkäufer bedienen Kunden und erfassen Premiumkunden.

# Adas Blumenladen

Alle können im Laden bezahlen, bar oder mit Kreditkarte. Bei Beträgen ab 100 Euro kann auch per Rechnung bezahlt werden. Sollte eine Rechnung nicht innerhalb von 14 Tagen beglichen werden, muss Frau König eine Mahnung versenden.

PremiumkundInnen können telefonisch eine Lieferung in Auftrag geben. Lieferungen können immer auf Rechnung bezahlt werden. Vor dem (kostenlosen) Versand durch einen Kurier muss die Rechnung beglichen sein.

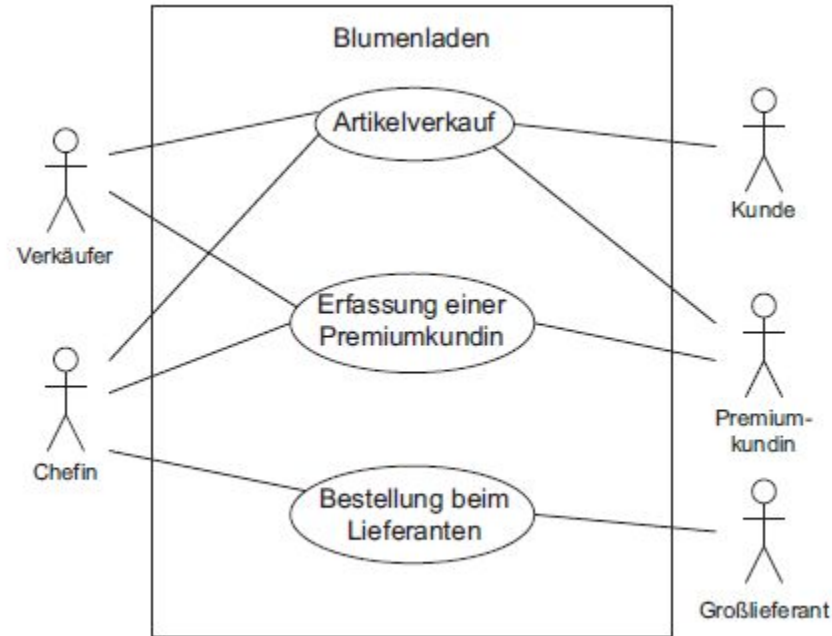
# Adas Blumenladen

Neben dem Verkauf muss auch das Lager gefüllt werden, sollten die Vorräte aufgebraucht sein. Dies kann entweder bei einer regelmäßigen Überprüfung auffallen, oder aber, wenn bei einem Kundenwunsch festgestellt wird, dass nicht genug Artikel vorhanden sind.

Die Bestellung muss durch die Chefin erfolgen, ein Verkäufer kann lediglich das frühestmögliche Lieferdatum erfragen.

Weiterhin kann die Chefin Artikel ins Sortiment aufnehmen, oder diese entfernen.

# Darstellung der Akteure und Anwendungsfälle (teilweise)



# Modularisierung

Am Anfang wollen wir die Software in verschiedene Teile unterteilen, die Unabhängig voneinander funktionieren. Diese Teilstücke nennen wir Module.

Einzelne Module sollten hierbei klar definierte Schnittstellen besitzen, über die sie Informationen austauschen.

Es sollte hierbei beachtet werden, dass zwei Module in fast jedem Projekt vorkommen: 'common' und 'UI' oder 'API'.

# UI/API

UI steht für user interface. Man bezeichnet damit im Normalfall die Oberfläche, mit der ein Nutzer manuell agiert.

Generell ist die UI eine der wichtigsten Komponenten. Sie sollte immer zuerst designed werden, da es sehr einfach ist, hier mit dem Kunden über seine Wünsche und Ansprüche zu reden. Außerdem gibt sie gute Ansätze für den Rest des Designs.

Zwei Mögliche Arten hier sind die GUI (Graphical user interface, Website, etc.) und CLI (command line interface, Python, etc.)

# UI/API

API steht für application programming interface.

Während eine (G)UI für die manuelle Interaktion gedacht ist, stellt eine API die programmatische Schnittstelle zur Anwendung her.

Wenn es sich bei der Anwendung um einen Service handelt (webservice, daemon), so besteht die API normalerweise aus endpoints, welche man ansprechen kann.

Wenn es sich bei der Anwendung um eine Library handelt, so besteht die API normalerweise aus einem header, welcher bestimmte Funktionen frei gibt.

# UI/API

Die Auswahl des Frontends (UI/API) ergibt sich normalerweise natürlich aus dem Anwendungsfall.

In dem Beispiel von Adas Blumenladen benötigen wir eine GUI, welche von allen Akteuren bedient werden kann, und automatisch alle möglichen Aktionen bereitstellt.

Dies kann in Form von Bildern geschehen. Heute ist es auch oft üblich, ein nicht-funktionales, aber interaktives GUI-Mock bereit zu stellen.

# Main menu Adas Blumenladen

Angemeldet als Ada König  
Rolle: Besitzer

Verkauf

Bestellung

Lagerbestand

Kunde registrieren

# Main menu Adas Blumenladen

In einem Design muss jedes Feld erklärt werden: Funktion, was es anzeigt, oder wo es hinführt.

Der Kunde muss eine möglichst vollständige Übersicht über seine Interaktion mit dem fertigen Programm erhalten

Angemeldet als Ada König  
Rolle: Besitzer

Verkauf

Bestellung

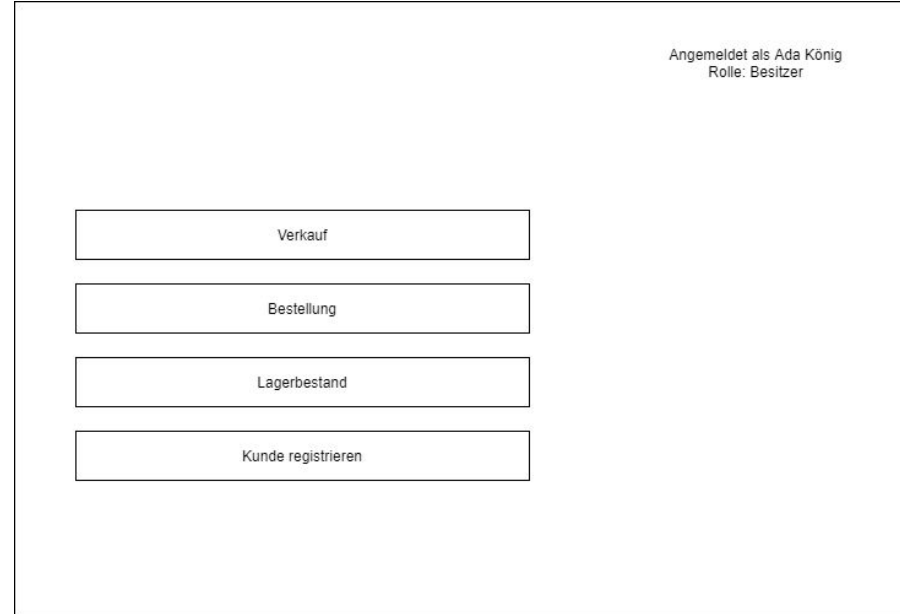
Lagerbestand

Kunde registrieren

The image shows a wireframe of a web application's main menu. In the top right corner, there is a user status section indicating the user is logged in as 'Ada König' with the role of 'Besitzer'. Below this, there are four rectangular buttons arranged vertically. The buttons are labeled 'Verkauf', 'Bestellung', 'Lagerbestand', and 'Kunde registrieren' from top to bottom.

# Main menu Adas Blumenladen

- Oben Rechts sieht man den angemeldeten Benutzer und seine Rolle (Abschnitt Rollen)
- Verkauf führt zum Verkaufsmenu
- Bestellung führt zum Bestellungsmenu
- Lagerbestand führt zum Bestandsmenu
- Kunde registrieren führt zum Registrierungsmenu



# Bestellungsmenu Adas Blumenladen

Das Bestellungsmenu sollte laut Anforderungsanalyse in verschiedenen Ausführungen enthalten sein.

- Die Chefin kann bestellen
- Andere VerkäuferInnen können lediglich die Verfügbarkeit abfragen

Hier ist es notwendig, zwei UI mocks zu erstellen, um den Unterschied zu verdeutlichen

# Bestellungsmenu Adas Blumenladen

## Bestellung

Angemeldet als Ada König  
Rolle: Besitzer

Eingabefeld Artikelnummer

Eingabefeld  
Anzahl

Prüfe Lieferdatum

Bestelle Artikel

## Bestellung

Angemeldet als Verkäuferin X  
Rolle: Verkäufer

Eingabefeld Artikelnummer

Eingabefeld  
Anzahl

Prüfe Lieferdatum

# Generelle Bemerkungen UI

- Die Technik, die benutzt wird, um die UI zu erstellen, ist für das Design irrelevant. Sollte man jedoch eine sehr ausgefeilt ausgearbeitete GUI erstellen wollen und führt diese vor, so sollte man sicher sein, dass die angewendete Technik dies unterstützt.
- Eine GUI sollte immer intuitiv sein, und keine große Einarbeitungszeit erfordern.
- Nutzerfehlerquellen sollten bereits im Design angesprochen und nach Möglichkeit vermieden werden.

# Nutzerfehler

“If your software can be used in a certain way, it will eventually be used in that way.”

Nutzerfehler sind eine der größten Fehlerquellen in der Benutzung von Software

Sie entstehen dadurch, dass dem Nutzer erlaubt wird, eine Software auf eine Art zu benutzen, die nicht für ihn gedacht war.

Nach Möglichkeit sollte jede Möglichkeit der unsachgemäßen Benutzung von vorneherein ausgeschlossen werden.

# Vermeidung eines Nutzerfehlers, Beispiel

In diesem Fall könnte der Hinweis auf aktives vermeiden eines Nutzerfehlers so gegeben werden:

Da nur der Besitzer bestellen kann, wird der Button für die Bestellung der Waren in der Verkäuferrolle nicht angezeigt. So kann der Verkäufer nicht fälschlicherweise annehmen, er hätte die Ware bestellt.

The image displays two screenshots of a web application interface for placing an order, titled 'Bestellung'. The top screenshot shows the interface for a user logged in as a 'Verkäufer' (Seller), while the bottom screenshot shows it for a user logged in as a 'Besitzer' (Owner).

**Top Screenshot (Verkäufer):**

- Title: Bestellung
- User Info: Angemeldet als Verkäuferin X, Rolle: Verkäufer
- Form Fields:
  - Eingabefeld Artikelnummer
  - Eingabefeld Anzahl
  - Prüfe Lieferdatum
- Missing Button: The 'Bestelle Artikel' button is not visible.

**Bottom Screenshot (Besitzer):**

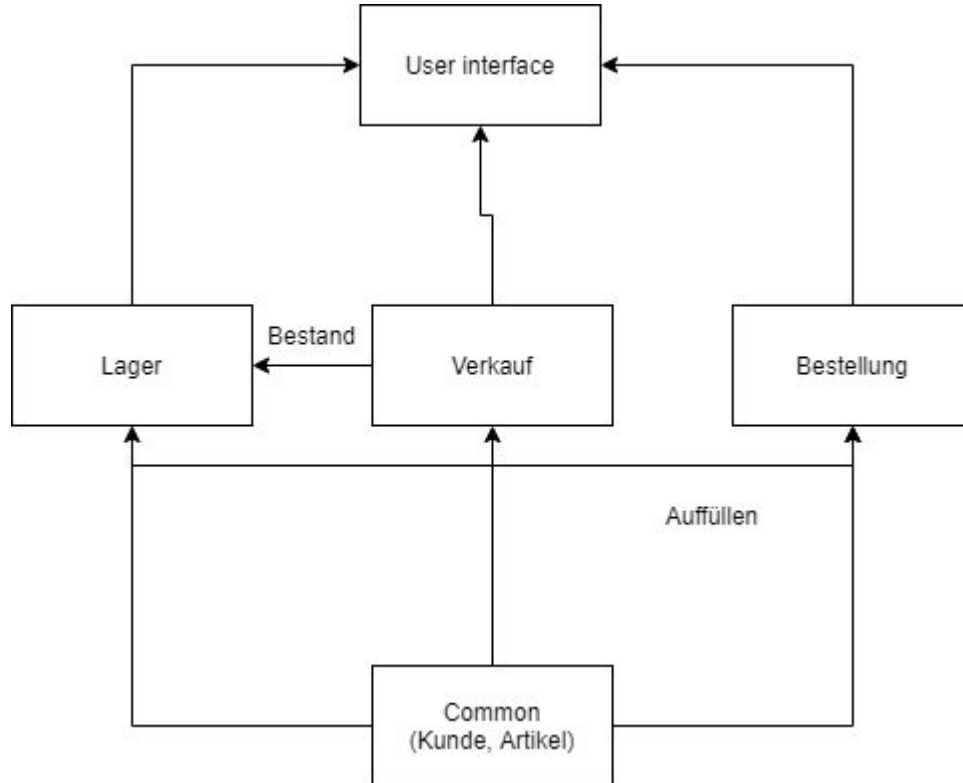
- Title: Bestellung
- User Info: Angemeldet als Ada König, Rolle: Besitzer
- Form Fields:
  - Eingabefeld Artikelnummer
  - Eingabefeld Anzahl
  - Prüfe Lieferdatum
  - Bestelle Artikel

# Modulübersicht

In der Modulübersicht beschreiben wir alle Module, die wir für die vollständige Entwicklung benötigen. Wir geben diese jeweils mit einer kurzen Beschreibung der Rolle an:

- **User interface**
  - Die graphische Oberfläche des Programms
- **Lagermodul**
  - Das Lagermodul enthält die Informationen über den Bestand im Lager. Es erzeugt Artikelklassen, und kann den Bestand angeben
- **Common**
  - Das Commonmodul enthält Klassen, die zur Kommunikation notwendig sind, und in mehreren Modulen benutzt werden

# Modulübersicht (teilweise)



# Modulübersicht

Es ist nicht notwendig, die Modulübersicht vor den einzelnen Modulen zu erzeugen.

Generell gilt: Das Designen der Software ist ein dynamischer Prozess, alle Stücke beeinflussen sich gegenseitig. Ein Modul kann die UI verändern, und genau so die Modulübersicht.

Lediglich der Code sollte erst nach abgeschlossenem Design gebaut werden, und dieses nicht mehr beeinflussen.

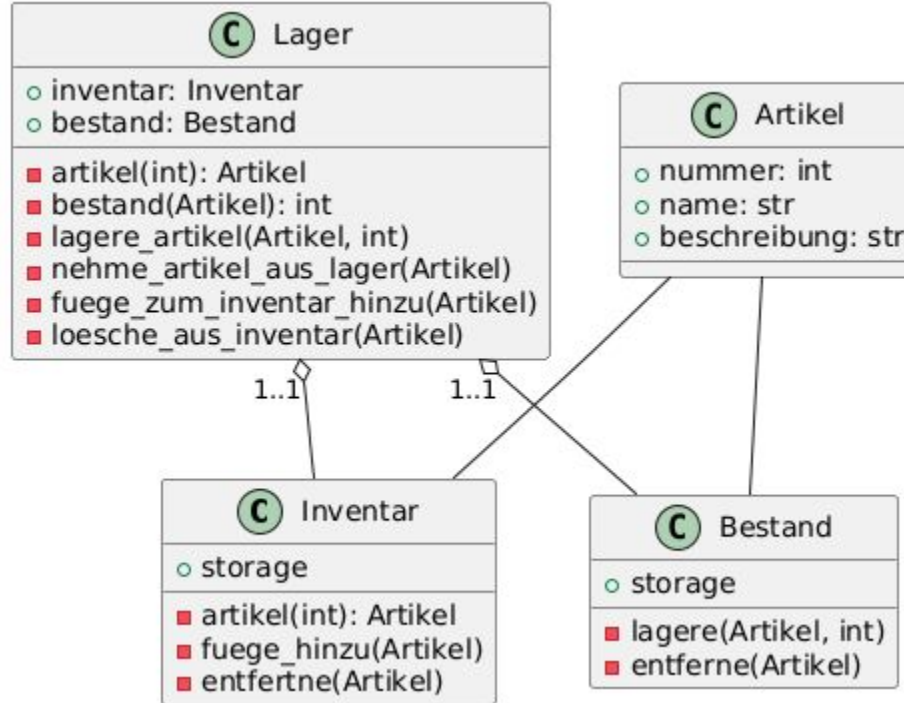
# Das Lagermodul

Das Lagermodul hat drei Aufgaben

- Zu einer Artikelnummer erzeugt es ein Objekt der Artikelklasse (Commonmodul)
- Es verwaltet die Übersicht aller Artikel
- Es verwaltet den Bestand im Lager

Wir wollen nun eine mögliche Übersicht angeben

# Klassenübersicht Lager



# Klassenübersicht Lager

Jede Klasse, und all ihre Felder und Funktionen, müssen nun vollständig dokumentiert werden.

Jede einzelne Funktion muss beschrieben werden, und jedes Feld muss beschrieben werden.

Evtl. sollten mögliche Alternativen angegeben werden.

# Klassenübersicht Lager

- Inventar enthält eine Liste von allen Artikeln, die zum Verkauf angeboten werden
- Bestand enthält den aktuellen Bestand eines jeden Artikels im Lager
- artikel erhält eine Artikelnummer, und gibt den entsprechenden Artikel zurück. Hierfür wird Inventar aufgerufen
- bestand erhält einen Artikel, und gibt den aktuellen Lagerbestand zurück. Hierfür wird Bestand aufgerufen.

| Lager                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| + Inventar: Inventar<br>+ Bestand: Bestand                                                                                                                                                                                   |
| + artikel(int): Artikel<br>+ bestand(Artikel): int<br>+ lagere_artikel(Artikel, int): void<br>+ nehme_artikel_aus_lager(Artikel): void<br>+ fuege_zum_inventar_hinzu(Artikel): void<br>+ loesche_aus_inventar(Artikel): void |

# Klassenübersicht Lager

- lagere\_artikel erhält einen Artikel und ein int anz. Die Anzahl der Artikel im Lager wird um anz erhöht.
  - Anmerkung: Die Namensgebung des Argumentes anz ist beabsichtigt. Sie dient der Lesbarkeit.
- nehme\_artikel\_aus\_lager erhält einen Artikel. Der Bestand des Artikels wird um 1 verringert.
- fuege\_zum\_inventar\_hinzu erhält einen Artikel. Der Artikel wird zum Inventar hinzugefügt.
- loesche\_aus\_inventar erhält einen Artikel. Der Artikel wird aus dem Inventar gelöscht.

| Lager                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| + Inventar: Inventar<br>+ Bestand: Bestand                                                                                                                                                                                   |
| + artikel(int): Artikel<br>+ bestand(Artikel): int<br>+ lagere_artikel(Artikel, int): void<br>+ nehme_artikel_aus_lager(Artikel): void<br>+ fuege_zum_inventar_hinzu(Artikel): void<br>+ loesche_aus_inventar(Artikel): void |

# Anmerkungen zur Erstellung der Klassen

- Lange Namen für Methoden sind vorzuziehen: Die Zeit von ANSI-C (8 chars pro Funktion) sind vorbei. Wenn möglich, sollte der Name einer Funktionen eine Idee davon geben, was diese tut.
- Es ist von Vorteil, Klassen an statt primitiver Werte zu übergeben. Wir könnten uns die Artikelklasse sparen, dies würde aber dazu führen, das wir evtl. unvollständige oder falsche Daten übergeben.
- Normalerweise enthalten Klassen bereits Annahmen, und diese gilt es zu dokumentieren. Wir haben z.B. die Annahme getroffen, dass die Artikelnummern manuell vergeben werden. Dies muss dokumentiert und mit dem Kunden besprochen werden.

# Dokumentation von Entscheidungen

Dinge, über die wir uns Gedanken machen müssen, und die im Design erwähnt werden müssen:

- Falls ein Artikel zum Inventar hinzugefügt wird, wird er auch mit Anzahl 0 zum Lager hinzugefügt?
- Muss das ausinventarisieren die Lagerbestände reduzieren?
- Was passiert, wenn ein Artikel aus dem Inventar gelöscht werden soll, der nicht vorhanden ist? Fehler oder nicht?

# Sollen wir einen Fehler werfen oder nicht

Generelles Mantra:

Fail early, fail fast

Es sollte immer versucht werden, mögliche Fehler so früh wie Möglich auszugeben.

Beispiel: Ein nicht-inventarisierter Artikel soll gelagert werden. Da nicht auszuschließen ist, dass ein Fehler in der Artikelbeschreibung gemacht wurde, sollte hier ein Fehler ausgegeben werden.

# Alternativen

Oft sind mehrere Möglichkeiten in einem Design funktional äquivalent, haben allerdings verschiedene Vor- und Nachteile.

Diese sollten erst aufgelistet und später, evtl. mit dem Kunden diskutiert werden. Oft ergibt sich aus diesem Gespräch die beste Alternative.

# Beispiel Alternativen: Storage

Wir geben verschiedene Möglichkeiten für Storage an:

- Datenbank
- Datei auf Rechner
- Cloudspeicher
- In memory

| Inventar                                                                 |
|--------------------------------------------------------------------------|
| + Storage                                                                |
| + artikel(int): Artikel<br>+ fuege_hinzu(Artikel)<br>+ entferne(Artikel) |

# Beispiel Alternativen: Storage

## Datenbank:

- Vorteile
  - Flexibel
  - Kann große Datenmengen verarbeiten
  - Konkurrierender Zugriff ist möglich
- Nachteile
  - Eine Datenbank muss extra aufgesetzt, verwaltet, und bezahlt werden

| Inventar                                                                 |
|--------------------------------------------------------------------------|
| + Storage                                                                |
| + artikel(int): Artikel<br>+ fuege_hinzu(Artikel)<br>+ entferne(Artikel) |

# Beispiel Alternativen: Storage

Datei auf Rechner:

- Vorteile
  - Einfach zu implementieren
  - Einfach zu sichern
- Nachteile
  - Kein konkurrierender Zugriff möglich
  - Falls Änderungen im Format gewünscht werden, so muss ein Konverter implementiert werden

| Inventar                                                                 |
|--------------------------------------------------------------------------|
| + Storage                                                                |
| + artikel(int): Artikel<br>+ fuege_hinzu(Artikel)<br>+ entferne(Artikel) |

# Beispiel Alternativen: Storage

## Cloudspeicher:

- Vorteile
  - Automatische Sicherung
  - Zugriff von mehreren Geräten
- Nachteile
  - Wie bei Datei auf Rechner

| Inventar                                                                 |
|--------------------------------------------------------------------------|
| + Storage                                                                |
| + artikel(int): Artikel<br>+ fuege_hinzu(Artikel)<br>+ entferne(Artikel) |

# Beispiel Alternativen: Storage

In memory:

- Vorteile
  - Kein Implementationsaufwand
- Nachteile
  - Bei Absturz des Systems werden die Daten gelöscht

| Inventar                                                                 |
|--------------------------------------------------------------------------|
| + Storage                                                                |
| + artikel(int): Artikel<br>+ fuege_hinzu(Artikel)<br>+ entferne(Artikel) |

# Das Commonmodul

Das Commonmodul enthält: Klassen, die in allen Modulen verwendet werden (in unserem Fall Rollen, Kunden), und Klassen, die zur Kommunikation notwendig sind (in unserem Fall Artikel).

Es ist wichtig, diese Klassen zu extrahieren, und nicht kreuz und quer im Code zu verteilen (Lesbarkeit, Erweiterbarkeit).

Man kann auch eine feinere Modulstruktur wählen, und ohne ein Commonmodul auskommen.

# Lager in python

```
from collections import defaultdict
import typing

class Artikel(typing.NamedTuple):
    nummer: int
    name: str
    beschreibung: str

class Bestand:
    def __init__(self):
        self.storage = defaultdict(int)
    def lagere(self, artikel, anzahl):
        self.storage[artikel.nummer] = self.storage[artikel.nummer] + anzahl
    def entferne(self, artikel):
        self.storage[artikel.nummer] = self.storage[artikel.nummer] - 1

class Inventar:
    def __init__(self):
        self.storage = dict()
    def artikel(self, nummer):
        if nummer not in self.storage.keys():
            raise ValueError('Artikel nicht im Inventar')
        return self.storage[nummer]
    def fuege_hinzu(self, artikel):
        self.storage[artikel.nummer] = artikel
    def entferne(self, artikel):
        del self.storage[artikel.nummer]
```

```
class Lager:
    def __init__(self):
        self.inventar = Inventar()
        self.bestand = Bestand()
    def artikel(self, nummer):
        return self.inventar.artikel(nummer)
    def lagere_artikel(self, artikel, nummer):
        self.bestand.lagere(artikel, nummer)
    def nehme_artikel_aus_lager(self, artikel):
        self.bestand.entferne(artikel)
    def fuege_zum_inventar_hinzu(self, artikel):
        self.inventar.fuege_hinzu(artikel)
    def loesche_aus_inventar(self, artikel):
        self.inventar.entferne(artikel)
```